

TECH LEAD & MENTORIA — GUIA DE REESTRUTURAÇÃO

De protótipo gerado por IA a arquitetura profissional

Um roteiro passo a passo para reestruturar o GestaoEstoque (Electron + SQLite) com banco de dados normalizado, arquitetura Main/Renderer correta, tipos ponta a ponta e uma estratégia de testes que cabe no seu tempo real.

Projeto: gestaoestoque (Electron Forge + Vite + React + Drizzle ORM + better-sqlite3)

Preparado para: Kennedy

Formato: guia conceitual — nenhuma linha de código de produção é entregue pronta; você escreve, eu reviso.

0. Sumário

1	Como vamos trabalhar juntos	Papéis e regras
2	Diagnóstico do banco atual	O que já vi no seu código
3	Ferramental: SQLite + Drizzle	Por que manter, o que trocar
4	Arquitetura Electron (Main × Renderer)	Camadas e pastas
5	Estratégia de testes	O que testar, quando, como
6	Plano de execução em fases	Ordem recomendada
7	Tarefa prática de hoje	O primeiro passo real
8	Perguntas de negócio	Preciso que você responda

1. Como vamos trabalhar juntos

Você pediu — corretamente — que eu não escreva o código final. Meu papel aqui é:

- **Mapear o problema** antes de qualquer linha ser apagada.
- **Explicar o "porquê"** de cada prática, não só o "como".
- **Fazer perguntas** sobre regras de negócio que só você sabe responder.
- **Revisar** o que você escrever, apontando desvios das boas práticas — não reescrever por você.

PRINCÍPIO GERAL

Você não está reescrevendo um app do zero — está fazendo uma **migração estrangulada** (strangler fig pattern): o projeto antigo continua rodando e sendo sua fonte de verdade de regras de negócio, enquanto o novo cresce ao lado, módulo por módulo, até substituir o antigo por completo. Isso evita o erro clássico de reescritas: parar 3 meses, não entregar nada, e descobrir requisitos esquecidos só no fim.

O que eu já observei no seu projeto (contexto real, não genérico)

Antes de escrever este guia, inspecionei a estrutura atual. Isso muda o diagnóstico de "teórico" para "aplicado ao seu caso":

- Você **já usa Drizzle ORM** (`drizzle-orm` , `drizzle-kit`) — não parte do zero. Boa notícia.
- Mas o schema Drizzle (`src/main/models/schema/`) descreve só **parte** do banco (`box` , `history` , `box_composition`). As tabelas de produção (`producao_ficha_diaria` , `producao_operador_diario` , `producao_registro_horario`) existem apenas como SQL cru dentro de `db.ts` — invisíveis para o ORM e para o TypeScript.
- O arquivo `src/main/models/db.ts` mistura três responsabilidades que deveriam estar separadas: conexão com o banco, *schema* (via `CREATE TABLE IF NOT EXISTS`) e *migração* (via blocos `try { ALTER TABLE ... } catch {}`).
- Existe um `db_query_main.js` solto na raiz do projeto — um script de debug ad-hoc que abre o banco diretamente para inspecionar dados. Sintoma de falta de ferramenta de administração adequada.
- Você já isola o acesso ao banco no processo **Main** e expõe uma API via `contextBridge` no `preload.ts` — isso é **exatamente certo** em termos de segurança do Electron. O ponto fraco não é "onde" o banco roda, é "como" o schema evolui e como os tipos trafegam.
- Existe também um **servidor Express interno** (`src/main/server/internalServer.ts`) rodando dentro do processo Main, aparentemente para uma janela de "produção geral" acessível por rede local. Isso significa que hoje você tem **dois canais de acesso ao domínio** (IPC e HTTP) — algo que a nova arquitetura de camadas (seção 4) precisa contemplar explicitamente.

Conclusão do diagnóstico rápido: seu problema não é "SQLite é ruim" nem "Drizzle é a ferramenta errada". O problema é que o **schema nunca foi tratado como a fonte única de verdade** — ele foi crescendo por remendos SQL diretamente no código de bootstrap do banco. É exatamente aí que vamos começar.

2. Diagnóstico do banco de dados

Antes de desenhar o banco novo, você precisa conseguir *ler* criticamente o banco antigo. Isso é uma habilidade, não uma tarefa única — vai usar isso toda vez que herdar um sistema legado na carreira.

2.1 Normalização — o que procurar

Sintoma	O que verificar	Exemplo já visto no seu schema
Dado duplicado / redundante	O mesmo valor aparece em várias tabelas e pode divergir?	<code>history.modelo</code> guarda texto livre do modelo, enquanto <code>box.model</code> também guarda o modelo do mesmo item. Se um dia o modelo do <code>box</code> mudar, o histórico já gravado fica "congelado" — isso é intencional (auditoria) ou é duplicação acidental? Você precisa decidir.
Tipo fraco para domínio fechado	Colunas <code>TEXT</code> que na verdade representam um conjunto fixo de valores	<code>box.step</code> , <code>box.location</code> , <code>history.typeOperation</code> são todos <code>TEXT</code> no SQL, mesmo tendo union types bem definidos no TypeScript (<code>ProductionStep</code> , <code>BoxLocation</code> , <code>HistoryOperation</code>). O banco não garante essas restrições — só o TypeScript garante, e só em tempo de compilação.
Ausência de integridade referencial real	Existem <code>FOREIGN KEY</code> de fato, e o banco está configurado para validá-las?	Você tem <code>.references()</code> no Drizzle e <code>REFERENCES</code> no SQL — bom. Mas <code>parentId</code> em <code>box</code> (auto-relacionamento para split/merge de lotes) não tem <code>references()</code> declarado no schema Drizzle. É intencional?
Migração via tentativa-e-erro	O schema evolui com <code>ALTER TABLE</code> dentro de blocos <code>try/catch</code> que engolem erro?	É exatamente o padrão em <code>db.ts</code> . Funciona, mas não é versionado, não é revertível, e não documenta <i>quando</i> cada mudança aconteceu nem <i>por quê</i> .
Regras de negócio "escondidas" em código, não no schema	Coisas como "todo <code>box</code> tem no máximo um <code>parent</code> ", mas pode ter vários registros em <code>box_composition</code> — isso está documentado em algum lugar formal?	Achei um comentário ótimo em <code>boxComposition.ts</code> explicando a regra de split/merge — isso é exatamente o tipo de conhecimento que precisa sobreviver à reescrita, de preferência virando um ADR (Architecture Decision Record) ou teste, não só um comentário.

2.2 Perguntas para guiar sua auditoria

Percorra cada tabela do banco atual e responda, por escrito, para cada uma:

1. Essa tabela representa uma **entidade** (algo que existe: uma caixa, um operador) ou um **evento** (algo que aconteceu: um histórico, uma movimentação)? Isso muda como ela deve se relacionar com o resto.
2. Quais colunas são, na prática, um **enum** disfarçado de `TEXT` ?
3. Existe algum campo que só faz sentido dependendo do valor de outro campo (ex: `volume` só importa se `origin = 'PRODUCTION'`)? Isso é sinal de que talvez a tabela devesse ser duas.
4. Quais índices existem hoje e **por que** foram criados — você consegue justificar cada um olhando as queries mais frequentes do app, ou foram criados "por precaução"?
5. Existe alguma tabela que na verdade deveria ser um relacionamento N:N com tabela de junção, mas está modelada como colunas soltas ou texto concatenado?

Ferramenta prática: gerar o diagrama ER do banco atual

Antes de desenhar o banco novo no papel, veja o banco atual como ele realmente é (não como você *acha* que ele é). Duas formas simples, sem instalar nada pesado:

1. Abra o arquivo `.sqlite` de produção (o caminho está em `drizzle.config.ts`) no **DB Browser for SQLite** (gratuito) e use a aba "Database Structure" — ele já mostra colunas, tipos e foreign keys declaradas.
2. Rode `sqlite3 caminho/bd_estoque.sqlite ".schema"` no terminal e cole a saída completa num arquivo `diagnostico-schema-atual.sql` — isso vira seu "antes" documentado, útil para comparar com o "depois" e para nossas conversas futuras.

3. Ferramental: banco de dados e ORM

3.1 SQLite continua sendo a escolha certa?

Sim, quase certamente. Para uma aplicação desktop Electron single-user (ou poucos usuários na mesma rede, dado o servidor interno que você já tem), SQLite é o padrão de mercado — usado por VS Code, WhatsApp Desktop, 1Password, entre outros. Trocar de banco não resolveria seus problemas atuais, porque o problema não é o motor de banco, é a **modelagem e a ausência de migrações versionadas**.

ATENÇÃO A UM PONTO ESPECÍFICO DO SEU PROJETO

Você tem um `internalServer.ts` com Express expondo funcionalidade por HTTP na rede local, além do IPC. SQLite lida mal com **múltiplos processos escrevendo ao mesmo tempo** sem WAL mode habilitado — e reparei que a linha `sqlite.pragma('journal_mode = WAL')` está comentada em `db.ts`. Isso é uma pergunta de arquitetura importante para a fase 3: só o processo Main deve tocar o arquivo `.sqlite` diretamente, nunca clientes remotos lendo o arquivo por rede (samba/drive compartilhado) — isso corrompe o banco. Se o Express só fala com o mesmo processo Main que já tem a conexão Drizzle, você está seguro; vale confirmar.

3.2 Drizzle ORM — manter, mas usar por completo

Drizzle é uma escolha sólida para Electron + SQLite: é "thin" (gera SQL previsível, sem mágica), tem excelente inferência de tipos TypeScript, e tem uma ferramenta de migração dedicada (**drizzle-kit**) que você já tem instalada mas não está usando meio a meio.

Opção	Quando faz sentido	Recomendação para seu caso
Drizzle ORM + drizzle-kit	Você quer SQL previsível, tipagem forte, migrations versionadas em arquivo, e já está familiarizado	Manter. Já está no projeto — o trabalho é <i>parar de contornar</i> o drizzle-kit com <code>sqlite.exec()</code> manual, e passar a usar <code>drizzle-kit generate</code> + <code>drizzle-kit migrate</code> de verdade.
Prisma	Times maiores, preferência por schema DSL próprio e client totalmente gerado	Migrar traria custo de reescrever tudo sem resolver o problema real (modelagem/migração indisciplinada). Não recomendo trocar só por trocar.
Kysely	Você quer só um query builder tipado, sem "ORM" de verdade	Alternativa válida, mas você perderia o <code>drizzle-kit</code> que já resolve migrations. Não traz ganho líquido aqui.
better-sqlite3 puro (sem ORM)	Controle total, performance máxima, times que preferem SQL cru revisado	Já é sua base (Drizzle roda em cima dele). Não precisa mudar.

3.3 O pilar que falta: migrations versionadas de verdade

O `drizzle-kit` já está configurado (`drizzle.config.ts`) mas o `db.ts` não usa o fluxo dele. O fluxo correto é:

1. Você edita o *schema* em TypeScript (única fonte de verdade).
2. Roda `drizzle-kit generate` — ele compara o schema com o histórico de migrations e gera um arquivo `.sql` novo, versionado, dentro de `/drizzle`.
3. Em runtime, no boot do app, você chama a função de migração do Drizzle (`migrate()`) do pacote `drizzle-orm/better-sqlite3/migrator`), que aplica só as migrations que o banco daquele usuário ainda não tem.

Isso substitui **todos** os blocos `try { ALTER TABLE... } catch {}` por arquivos `.sql` numerados, revisáveis em code review, e com histórico no Git.

POR QUE ISSO IMPORTA NA PRÁTICA

Hoje, se você reinstalar o app numa máquina nova, o `db.ts` roda tudo do zero e o banco fica correto "por sorte" (porque os `CREATE TABLE IF NOT EXISTS` já nascem com o schema final). Mas para um usuário que já tem o app instalado há meses, o banco dele só chega ao estado atual passando por uma sequência de `ALTER TABLE` silenciosos. Isso é frágil e impossível de auditar. Migrations versionadas resolvem os dois casos com o mesmo mecanismo.

4. Arquitetura Electron: Main vs. Renderer

4.1 O que você já acerta

Seu `preload.ts` usa `contextBridge.exposeInMainWorld` em vez de expor `ipcRenderer` diretamente — isso é a prática de segurança recomendada oficialmente pelo Electron (`contextIsolation: true`). O banco só é tocado dentro de `src/main`. Mantenha esse princípio como regra inegociável: **o Renderer nunca importa `better-sqlite3` ou `drizzle-orm` diretamente, nunca.**

4.2 O problema: camadas dentro do Main estão misturadas

Olhando `src/main/controllers` e `src/main/services`, você já tem o começo de uma separação (controller → service). O que falta é uma camada explícita de **acesso a dados** (repository) entre o service e o Drizzle, e um contrato de tipos compartilhado real entre Main e Renderer.

Camadas recomendadas dentro do processo Main

```
src/main/  
  db/  
    schema/      <- única fonte de verdade da estrutura (Drizzle)  
    migrations/  <- gerado por drizzle-kit, nunca editado à mão  
    client.ts     <- só abre a conexão e roda migrate()  
  repositories/  <- ÚNICO lugar que escreve queries Drizzle  
    boxRepository.ts  
    historyRepository.ts  
  services/      <- regras de negócio, orquestra repositories,  
                  não sabe o que é SQL  
  ipc/           <- registra os handlers ipcMain.handle(...)  
                  e SÓ chama services
```

Regra de dependência (a parte mais importante)

A informação só pode fluir em uma direção. Isso evita que "regra de negócio" vaze para dentro de queries SQL, e que SQL vaze para dentro de controllers:

```
ipc/ (handlers) → services/ (regras de negócio) → repositories/ (SQL/Drizzle) → db/ (sche
```

Um `service` nunca deveria importar `drizzle-orm` diretamente — ele chama métodos do `repository`, como `boxRepository.findByStep('Montagem')`. Isso é o que te dá **testabilidade**: em teste, você troca o repository real por um fake em memória, sem precisar de SQLite de verdade.

4.3 O contrato de tipos entre Main e Renderer

Hoje o `preload.ts` usa `data: any` em quase toda função. Isso anula a vantagem de usar TypeScript — o Renderer pode mandar qualquer coisa e só vai quebrar em runtime, dentro do Main.

O padrão de mercado para isso em Electron+TS é: definir os tipos de **request/response de cada canal IPC** uma única vez em `src/shared/` (pasta que você já tem!) e importar esse mesmo tipo dos dois lados — no handler do Main e no `preload.ts`. Assim um erro de contrato quebra a *compilação*, não a produção.

4.4 E o servidor Express interno?

Isso merece uma decisão consciente, não incidental. O Express que você tem hoje em `internalServer.ts` deveria, na arquitetura nova, chamar exatamente os mesmos `services/` que os handlers de IPC chamam — nunca acessar `repositories/` ou o banco por conta própria. Dessa forma você tem dois "adaptadores de entrada" (IPC e HTTP) para o mesmo núcleo de regras de negócio, que é exatamente o desenho de uma **arquitetura hexagonal / ports & adapters** — o nome formal do que seu instinto já começou a fazer ao separar controllers de services.

P: Isso é intencional — outras estações da fábrica acessam essa janela "geral" por rede local enquanto o app roda em uma máquina central?

Preciso que você confirme isso antes da fase de arquitetura, porque muda se o SQLite precisa de WAL mode ligado e se algum dia você vai precisar de um banco cliente-servidor de verdade (Postgres) em vez de SQLite.

5. Estratégia de testes

5.1 Quando começar: não espere terminar a reestruturação

O erro mais comum é achar que "primeiro arrumo tudo, depois escrevo teste". Na prática, os testes são a ferramenta que te dá **confiança de que a versão nova faz o mesmo que a versão antiga** — e você precisa dessa confiança desde o primeiro módulo migrado, não no final.

5.2 Ordem recomendada de introdução de testes

Etapa	Tipo de teste	O que cobre no seu caso
1	Testes de repository (integração leve)	Sobem um SQLite em memória (<code>:memory:</code>), rodam as migrations reais do drizzle-kit, e testam se <code>boxRepository.create(...)</code> realmente persiste e busca certo. Rápidos (SQLite em memória é praticamente instantâneo) e pegam erro de schema/SQL cedo.
2	Testes de service (unitário)	Regra de negócio pura — ex: "um <code>box</code> com <code>step = 'Concluida'</code> não pode receber <code>startStep</code> de novo", ou a lógica de split/merge de <code>box_composition</code> . Usam um repository <i>fake</i> (em memória, sem SQLite), então rodam em milissegundos.
3	Testes de contrato IPC	Garantem que cada handler registrado em <code>ipc/</code> valida entrada e devolve o formato combinado em <code>shared/types.ts</code> .
4	E2E (Playwright para Electron)	Só depois que o núcleo estiver estável — cobre os fluxos críticos ponta a ponta (criar caixa, mover de etapa, expedir). Poucos testes, só os caminhos mais valiosos: E2E é lento e caro de manter.

REGRA PRÁTICA DE PRIORIZAÇÃO

Escreva o teste de repository/service **toda vez que migrar uma função de `db.ts` ou de um `*Services.ts` atual** para a nova arquitetura. Isso vira seu critério de "pronto": uma funcionalidade só está migrada quando tem teste comprovando que o comportamento é igual (ou deliberadamente melhor, com a mudança documentada).

5.3 Ferramentas recomendadas

- **Vitest** — você já usa Vite no projeto (forge + vite), então Vitest se integra sem configuração extra e é hoje o padrão de facto para projetos Vite/TS.
- **better-sqlite3 com `:memory:`** para os testes de repository — sem mocks complicados, sem Docker, roda local e no CI igual.

- **Playwright** (com o driver Electron dele) para os poucos testes E2E do final da fase.

6. Plano de execução em fases

Sugestão de sequência — cada fase entrega algo funcional, nada fica "pela metade" por semanas:

Fase 0 — Preparação (sem escrever código de produção ainda)

- ☐ Copiar o projeto atual para uma pasta de referência (ex: `gestaoestoque-legado/`), fora do novo repositório Git.
- ☐ Criar o novo repositório Git limpo.
- ☐ Documentar o "antes": diagrama ER atual + lista de todas as regras de negócio que você já sabe (o comentário sobre split/merge em `boxComposition.ts` é um ótimo ponto de partida).
- ☐ Definir, junto comigo, o desenho do schema novo (seção 7 traz a primeira tarefa disso).

Fase 1 — Fundação: banco de dados

- ☐ Modelar o schema novo em Drizzle, incluindo as tabelas de produção que hoje estão fora do ORM.
- ☐ Gerar a primeira migration real com `drizzle-kit generate`.
- ☐ Escrever os primeiros testes de repository contra SQLite em memória.

Fase 2 — Camada de dados e serviços (por módulo, um de cada vez)

- ☐ Escolher **um módulo** por vez (sugiro começar por `box / history`, que é o núcleo do estoque) e migrar repository → service → handler IPC.
- ☐ Repita para `box_composition`, depois para o módulo de produção.

Fase 3 — Renderer e contrato de tipos

- ☐ Tipar o `preload.ts` e os hooks/páginas do React contra os tipos de `shared/`.

Fase 4 — Testes E2E dos fluxos críticos + corte do legado

- ☐ Playwright cobrindo os 3-5 fluxos mais importantes do dia a dia da fábrica.
- ☐ Só desligar o app antigo depois que o novo cobrir 100% do que ele fazia.

7. A primeira tarefa prática de hoje

Tarefa: Auditoria escrita do schema atual (sem código ainda)

Antes de desenhar qualquer coisa nova, quero que você produza um documento simples (pode ser um `.md` no novo repositório) respondendo, **para cada tabela existente** (`box`, `history`, `box_composition`, `producao_ficha_diaria`, `producao_operador_diario`, `producao_registro_horario`):

1. Em uma frase: o que essa tabela representa no mundo real da fábrica?
2. Quais colunas são, na verdade, um enum fechado (liste os valores possíveis)?
3. Quais relacionamentos existem com outras tabelas, e a cardinalidade de cada um (1:N, N:N)?
4. Existe algum dado derivável de outro (ex: algo que poderia ser calculado em vez de armazenado)?
5. O que te incomoda nessa tabela hoje — o que já causou bug ou dor de manutenção?

Esse documento vira a base da nossa próxima conversa: eu vou revisar suas respostas, fazer perguntas em cima delas, e só então começamos a desenhar o schema novo juntos — com você desenhando, eu perguntando "e se..."

NÃO FAÇA AINDA

Não instale nada novo, não apague nada, não crie o repositório novo até fechar esse documento. A tentação de "só configurar o Prisma rapidinho" é o jeito mais comum de gastar um fim de semana e não avançar no que importa: entender o domínio.

8. Perguntas de negócio para nossa próxima conversa

Estas perguntas não têm resposta "certa" genérica — só você sabe a regra real da sua fábrica. Pense nelas enquanto faz a tarefa da seção 7:

P: Uma `box` pode pertencer a mais de um "lote" ao mesmo tempo, ou a relação `parent/composition` é estritamente uma árvore (cada caixa tem no máximo uma origem direta)?

P: O campo `history.modelo` duplicar o `box.modelo` é proposital (snapshot para auditoria) ou seria melhor resolver isso com um JOIN?

P: O servidor Express interno (janela "produção geral") é acessado por outras máquinas na rede da fábrica enquanto o SQLite está aberto no processo Main? Quantas estações, tipicamente, ao mesmo tempo?

P: Existe alguma tabela ou fluxo que você sabe que está "errado" mas nunca teve tempo de corrigir — algo que já causou retrabalho manual ou dado inconsistente?

P: O app precisa funcionar 100% offline sempre, ou já existe (ou existirá) a necessidade de sincronizar dados entre máquinas/um servidor central no futuro?